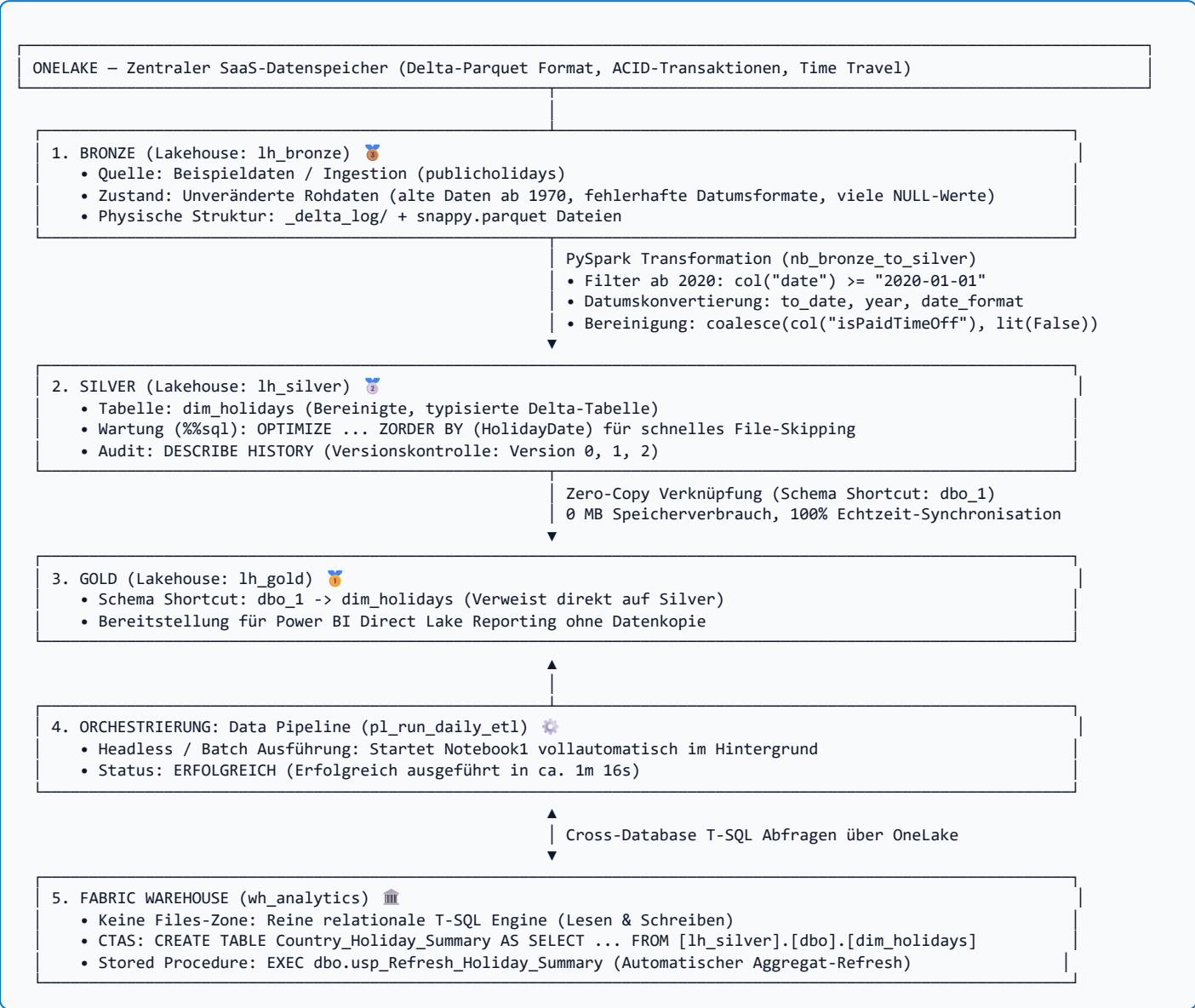


Microsoft Fabric Data Engineering — Master-Wegweiser (DP700_Lab)

Komplettanleitung zum selbstständigen Nachbauen von A bis Z

Medallion Architecture, PySpark, Delta Optimization, Zero-Copy Shortcuts, Data Pipelines & Fabric Warehouse T-SQL

TEIL 1: Das Gesamtsystem auf einen Blick (End-to-End Architektur)



TEIL 2: Das Ingenieur-Lexikon (Die wichtigsten Begriffe auf Deutsch & Russisch)

Begriff	Was bedeutet das in der Praxis?	Relevanz für Data Engineers & Architektur
OneLake	Ein zentraler Data Lake für das gesamte Unternehmen („OneDrive für Daten“). Basiert auf ADLS Gen2.	Zentraler Speicher: Alle Engines (Spark, SQL, BI) greifen ohne Datenkopien auf denselben Speicher zu.
Lakehouse vs Warehouse	Lakehouse: Python/Spark + Files + Tables (SQL nur lesend). Warehouse: Reines T-SQL, Lese- und Schreibzugriff, Stored Procedures, ACID-Transaktionen.	Klare Architektur-Trennung: Lakehouse für Spark & Transformation (Bronze/Silver); Warehouse für relationales Serving und Datamarts.
Delta Lake (_delta_log)	Open-Source-Speicherformat. Kombiniert Parquet mit einem Transaktionsprotokoll. Ermöglicht ACID, Versionierung und Time Travel.	Enterprise-Speicher: Parquet + Transaktionslog. Garantiert ACID-Zuverlässigkeit, Time Travel und Versionierung.
Z-Order & Optimize	Fasst kleine Dateien zusammen (Bin-Packing) und sortiert Daten physisch nach Spalten für <i>File Skipping</i> .	Datei-Optimierung: Beseitigt Small-File-Probleme und beschleunigt Lesezugriffe durch File Skipping um Faktoren.
Schema Shortcut (Verknüpfung)	Ein Zeiger auf ein Schema oder eine Tabelle in einem anderen Lakehouse. 0 Byte kopiert.	Zero-Copy Architektur: Tabellen werden in Gold logisch verknüpft – spart Speicherkosten und verhindert Desynchronisation.
Headless Pipeline	Automatischer Batch-Lauf im Hintergrund. Startet Notebooks ohne Benutzeroberfläche nach Zeitplan.	Batch-Orchestrierung: Zuverlässige, zeitgesteuerte ETL-Ausführung im Hintergrund via Data Factory.
CTAS	<code>CREATE TABLE ... AS SELECT</code> : Erstellt und befüllt eine neue Tabelle in einem einzigen T-SQL Befehl.	Hochperformante Materialisierung vorberechneter Aggregationstabellen für Endanwender.
Stored Procedure	Gespeicherte T-SQL Prozedur im Warehouse. Kann mit <code>EXEC name;</code> jederzeit aufgerufen werden.	Serverseitige Kapselung von ETL-Aktualisierungslogiken mit ACID-Transaktionssicherheit.

TEIL 3: Schritt-für-Schritt Anleitung zum selbstständigen Nachbauen

1 Workspace anlegen (DP700_Lab)

1. Links auf *Arbeitsbereiche* → + *Neuer Arbeitsbereich* klicken.
2. Name: `DP700_Lab_Practice`.
3. Unter *Erweitert* → Lizenzierungsmodus: **Fabric-Testversion (Trial)** auswählen → *Übernehmen*. (Diamant-Symbol  erscheint).

2 Bronze-Schicht & Rohdaten anlegen

1. Im Workspace auf + *Neues Element* → **Lakehouse** klicken → Name: `lh_bronze`.
2. In der Mitte auf **Mit Beispieldaten beginnen** klicken → Beispieldatensatz (Public Holidays) laden.
3. Ergebnis prüfen: In `lh_bronze` → **Tables** → `dbo` → `publicholidays` ist sichtbar.

3 Silver-Schicht & PySpark Transformation

1. Im Workspace auf + *Neues Element* → **Lakehouse** → Name: `lh_silver`.
2. Oben rechts auf *Daten analysieren mit* → **Neues Notebook** klicken → Name oben links in `nb_bronze_to_silver` ändern.
3. Links im Explorer auf + *Datenelemente hinzufügen* → *Vorhandenes Lakehouse* → `lh_bronze` hinzufügen.
4. **Zelle 1 (PySpark Transformation):**

```
from pyspark.sql.functions import col, to_date, year, date_format, coalesce, lit

# 1. Aus Bronze einlesen
df_raw = spark.read.table("lh_bronze.dbo.publicholidays")

# 2. Bereinigen & Typisieren
df_silver = df_raw \
    .filter(col("date") >= "2020-01-01") \
    .withColumn("HolidayDate", to_date(col("date"))) \
    .withColumn("Year", year(col("HolidayDate"))) \
    .withColumn("DayOfWeek", date_format(col("HolidayDate"), "EEEE")) \
    .withColumn("IsPaid", coalesce(col("isPaidTimeOff"), lit(False))) \
    .select(
        col("countryRegionCode").alias("CountryCode"),
        col("countryOrRegion").alias("CountryName"),
        col("holidayName").alias("HolidayName"),
        col("HolidayDate"),
        col("Year"),
        col("DayOfWeek"),
        col("IsPaid")
    )

# 3. In Silver als Delta speichern
df_silver.write.format("delta").mode("overwrite").saveAsTable("lh_silver.dbo.dim_holidays")
print("Silver erfolgreich geschrieben!")
```

5. Zelle 2 (Delta Maintenance & Z-Order):

```
%%sql
OPTIMIZE lh_silver.dbo.dim_holidays ZORDER BY (HolidayDate);
DESCRIBE HISTORY lh_silver.dbo.dim_holidays;
```

6. Ausführen (Play ►). Ergebnis: `dim_holidays` existiert in `lh_silver` mit 3 Versionen im Log.

4 Gold-Schicht & Zero-Copy Schema Shortcut

1. Im Workspace auf + *Neues Element* → **Lakehouse** → Name: `lh_gold`.
2. Links im Explorer auf **Tables** die drei Punkte ... anklicken → **Neue Schemaverknüpfung**.
3. *Microsoft OneLake* → `lh_silver` → Haken bei der Checkbox **dbo** setzen → *Weiter* → *Erstellen*.
4. Ergebnis: In `lh_gold` erscheint `dbo_1` mit der verlinkten Tabelle `dim_holidays` (Ketten-Symbol , 0 MB Speicher).

5 Orchestrierung per Data Pipeline (Automatisierung)

1. Im Workspace auf + *Neues Element* → **Datenpipeline** → Name: `pl_run_daily_etl`.
2. Im Menü oben auf **Notebook** klicken → Unten bei *Einstellungen* das Notizbuch `nb_bronze_to_silver` auswählen.
3. Oben links auf *Speichern*  und dann auf **Ausführen** ► klicken.
4. Ergebnis: Pipeline schließt mit Status **Erfolgreich** (grüner Haken) in ca. 1 Minute ab.

6 Fabric Warehouse & T-SQL Cross-Database Queries

1. Im Workspace auf + *Neues Element* → **Warehouse** → Name: `wh_analytics`.
2. Links im Explorer auf + **Warehouses** klicken → `lh_silver` ankreuzen (damit die Tabellen sichtbar werden).
3. Oben auf **Neue SQL-Abfrage** klicken → CTAS-Tabelle erstellen:

```
CREATE TABLE dbo.Country_Holiday_Summary AS
SELECT
    CountryCode, CountryName,
    COUNT(*) AS TotalHolidays,
    SUM(CAST(IsPaid AS INT)) AS PaidHolidays,
    MIN(HolidayDate) AS FirstDate,
    MAX(HolidayDate) AS LastDate
FROM [lh_silver].[dbo].[dim_holidays]
GROUP BY CountryCode, CountryName;
```

4. Stored Procedure erstellen:

```
CREATE OR ALTER PROCEDURE dbo.usp_Refresh_Holiday_Summary
AS
BEGIN
    TRUNCATE TABLE dbo.Country_Holiday_Summary;
    INSERT INTO dbo.Country_Holiday_Summary
    SELECT
        CountryCode, CountryName,
        COUNT(*) AS TotalHolidays,
        SUM(CAST(IsPaid AS INT)) AS PaidHolidays,
        MIN(HolidayDate) AS FirstDate,
        MAX(HolidayDate) AS LastDate
    FROM [lh_silver].[dbo].[dim_holidays]
    GROUP BY CountryCode, CountryName;
END;
```

5. Testausführung: `EXEC dbo.usp_Refresh_Holiday_Summary;` → 38 Zeilen aktualisiert in 2 Sekunden!

Fazit für die Praxis: Mit diesen 6 Schritten beherrschst du den gesamten Datenlebenszyklus in Microsoft Fabric: von der Rohdatenaufnahme über die Spark-Transformation bis zur relationalen Bereitstellung im Data Warehouse!